SOFTWARE ENGINEERING AND ARCHITECTURE

Self Learning Topic:1

1. Comparison of Agile and DevOps

Agile: Agile is a flexible software development approach based on iterative development, continuous feedback, customer collaboration, and adapting to changing requirements. It delivers software in small, manageable increments.

DevOps: DevOps combines Development and Operations to improve software delivery. It emphasizes automation, continuous integration and delivery, testing, deployment, monitoring, and collaboration between teams.

Agile vs DevOps

Aspect	Agile	DevOps
Focus	Focuses on flexible and iterative software development.	Focuses on continuous software delivery and operations.
Goal	Delivers working software frequently based on customer needs.	Delivers software quickly and reliably through automation.
Development	Uses short development cycles called sprints or iterations.	Uses a continuous process from development to deployment.
Teams	Encourages collaboration between developers, testers, and customers.	Encourages collaboration between development and operations teams.
Feedback	Uses regular customer feedback to improve the product.	Uses customer and system feedback for continuous improvement.
Testing	Testing is performed throughout development.	Testing is automated and integrated into the delivery process.
Automation	Automation is useful but not essential.	Automation is a major part of DevOps.
Deployment	Usually takes place after an iteration or release cycle.	Can be frequent and automated.
Tools	JIRA, Asana, and Trello.	Jenkins, Docker, and Kubernetes.
Main Benefit	Provides flexibility and faster response to requirements.	Provides faster and more reliable software delivery.

2. Case Study of JIRA and Asana in Real Projects

2.1 Case Study: JIRA – Spotify

Company: Spotify

Industry: Music Streaming

Spotify uses Agile practices and Jira to manage software development tasks and coordinate work among different teams.

Spotify used JIRA to:

- Plan and prioritize development tasks.
- Track bugs and project progress.
- Manage requirements and user stories.
- Coordinate work between development and testing teams.
- Improve visibility of ongoing development activities.
- Support Agile development through iterations and continuous feedback.

Result: JIRA helped Spotify organize development activities, track project progress, and improve collaboration between teams.

Conclusion: This case shows how JIRA can support Agile software development by managing requirements, tasks, bugs, and team collaboration.

2.2 Case Study: Asana – Airbnb

Company: Airbnb

Industry: Online Travel and Hospitality

Airbnb uses work-management practices to coordinate projects involving product, engineering, design, marketing, and other business teams. Asana can be used to provide a centralized view of tasks, deadlines, responsibilities, and project progress.

Airbnb used Asana to:

- Organize projects and tasks across teams.
- Assign responsibilities and deadlines.
- Improve communication between technical and business teams.
- Track project progress and upcoming activities.

- Create structured workflows using projects and templates.
- Improve visibility of work across departments.

Result: Asana helped teams organize work in a centralized system, improve project visibility, and coordinate activities across different departments.

Conclusion: This case demonstrates how Asana can support large organizations by organizing projects, assigning work, tracking progress, and improving collaboration.

3. Writing Effective User Stories and Acceptance Criteria

3.1 User Stories

A user story is a short description of a requirement written from the user's point of view. It explains who needs a feature, what they need, and why they need it.

Basic Format:

As a [user], I want [feature], so that [benefit].

Example:

As a customer, I want to add products to my shopping cart so that I can purchase them later.

Characteristics of an Effective User Story

- Clear and easy to understand
- Focused on the user's needs
- Provides business or user value
- Small enough to implement
- Can be tested
- Allows discussion between the team and stakeholders

3.2 Acceptance Criteria

Acceptance criteria are the conditions that a feature must satisfy to be considered complete and accepted by the customer or product owner.

Example

User Story: As a customer, I want to add a product to my shopping cart so that I can purchase it later.

Acceptance Criteria:

1. The customer can click the Add to Cart button.

2. The selected product appears in the shopping cart.
3. The cart quantity is updated when the same product is added again.
4. The total price is updated automatically.
5. The customer receives a confirmation that the product was added successfully.

Importance of Acceptance Criteria

Acceptance criteria make requirements clear, measurable, and testable. They also help developers understand what needs to be implemented and help testers determine whether the feature works correctly.

4. Advanced Requirement Elicitation Techniques

Requirement elicitation is the process of collecting and understanding the needs, expectations, and requirements of stakeholders before developing a software system. Advanced techniques help identify detailed and sometimes hidden requirements.

4.1 Interviews

An interview is a direct discussion between a requirements analyst and stakeholders to understand their needs, problems, and expectations.

Types

- Structured: Uses a fixed set of questions.
- Semi-structured: Uses prepared questions with flexibility for additional questions.
- Unstructured: Allows an open-ended discussion.

Example

For a hospital management system, an analyst may interview doctors, nurses, and receptionists to understand their daily tasks and system requirements.

Advantages

- Provides detailed information.
- Allows follow-up questions.
- Helps understand stakeholder expectations.

4.2 Ethnography

Ethnography is a technique in which the analyst observes users in their actual working environment to understand how they perform their tasks. Instead of only asking users what they do, the analyst observes their real activities and identifies requirements that users may not mention during interviews.

Example

For a warehouse management system, an analyst may observe workers while they scan products, update inventory, and move packages. This can reveal requirements such as mobile access, barcode scanning, and support for poor network connectivity.

Advantages

- Identifies hidden requirements.
- Provides understanding of real-world workflows.
- Captures actual user behavior.
- Useful for complex working environments.

Interviews vs Ethnography

Aspect	Interviews	Ethnography
Method	Collects requirements by directly asking stakeholders questions.	Collects requirements by observing users in their workplace.
Understanding	Helps understand what users say they need.	Helps understand what users actually do.
Time	Faster and easier to conduct.	Usually requires more time and observation.
Useful for	Understanding opinions and expectations.	Discovering hidden and practical requirements.

5. Requirement Traceability Matrix (RTM)

A Requirement Traceability Matrix (RTM) is a structured document used to track software requirements throughout the development lifecycle. It links each requirement with its corresponding design, implementation, and test cases.

The main purpose of RTM is to ensure that all requirements are implemented and tested and that no requirement is missed.

Requirement ID	Requirement	Design	Test Case	Status
R01	User login functionality	D01	TC01	Completed
R02	Product search functionality	D02	TC02	Completed

R03	Online payment functionality	D03	TC03	In Progress
R04	Order history functionality	D04	TC04	Not Started

Benefits of RTM

- Ensures complete requirement coverage.
- Helps identify missing requirements.
- Connects requirements with test cases.
- Makes requirement changes easier to track.
- Improves project quality and documentation.

6. Requirement Management Tools – IBM DOORS

IBM DOORS (Dynamic Object-Oriented Requirements System) is a requirement management tool used to manage requirements in complex and large-scale projects. It helps teams document, organize, trace, review, and control requirements throughout the software development lifecycle.

Key Features of IBM DOORS

Feature	Description
Requirement Management	Allows requirements to be created, organized, and maintained.
Traceability	Links requirements with design, development, and testing activities.
Change Management	Tracks changes and helps identify their impact.
Version Control	Maintains the history of requirement modifications.
Collaboration	Allows stakeholders to review and discuss requirements.
Reporting	Provides structured requirement documents and reports.

Example

In an automotive braking system project, a requirement such as “The braking system shall respond within the specified time” can be recorded in IBM DOORS.

The requirement can then be linked to:

Requirement → Design → Implementation → Test Case → Test Result

If the requirement changes, these links help the team identify the affected design components and test cases.

Conclusion: Agile, DevOps, and IBM DOORS help improve software development by promoting flexibility, collaboration, automation, and effective requirements management. Together, they support better quality, traceability, and efficient delivery of complex projects.